



Práctica 4. Conexión y programación de receptor infrarrojo y LCD a través del I2C con el NodeMCU.

Profesor: Juan Moreno García

Índice

1. Conexión receptor y mando infrarrojo.
2. Conexión LCD.
3. Receptor y mando infrarrojo junto con LCD .
4. Mejoras.

1. Receptor y mando infrarrojo.

- * Circuito inter-integrado (I2C - Inter-Integrated Circuit) es **un bus serie de datos que se utiliza para la comunicación entre diferentes partes de un circuito.**
- * Este sistema de comunicación utiliza **dos líneas de transmisión:** SDA (datos serie) y SCL (reloj serie).
- * Al ser un bus **puede ser compartido entre varios dispositivos.**
- * En esta práctica se conectará un **keypad y el nodeMCU.**
- * El funcionamiento del keypad se puede ver en la siguiente web: **<https://www.luisllamas.es/arduino-teclado-matricial/>**

1. Receptor y mando infrarrojo.

```
#include <IRremoteESP8266.h>  
#include <IRrecv.h>
```

INSTALACIÓN DE LIBRERÍAS

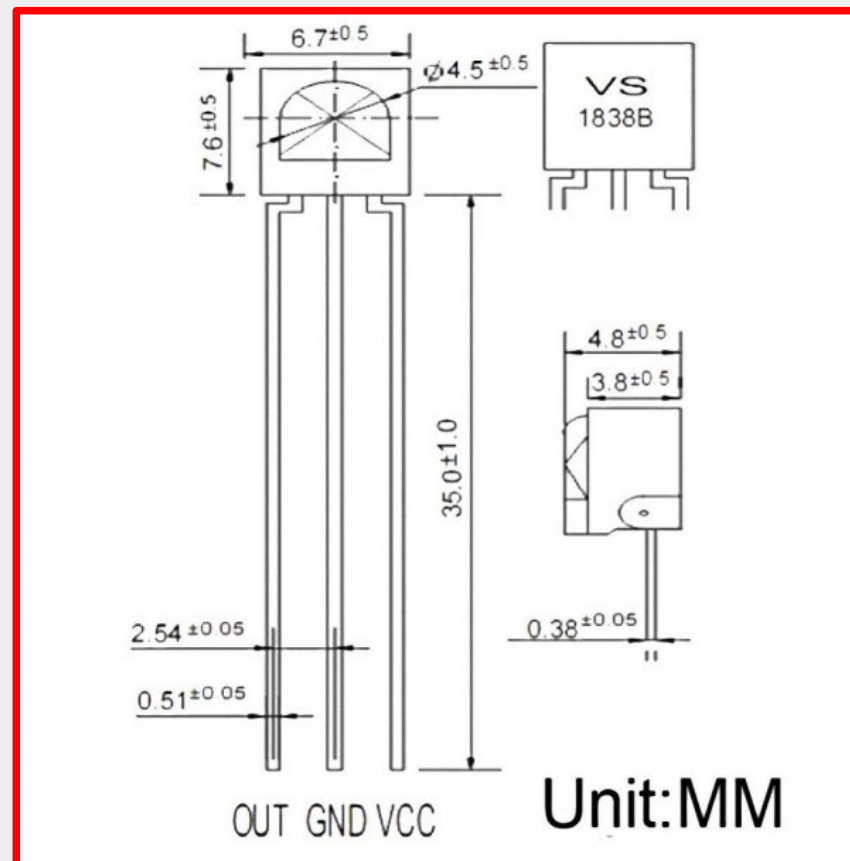
- * Para poder utilizar el mando infrarrojo se necesita un receptor infrarrojo conectado al nodeMCU.
- * Para poder programarlo es necesario bajar la librería IRremoteESP8266. Bajar de la dirección <https://github.com/markszabo/IRremoteESP8266> el archivo zip que contiene la librería y copiar la carpeta que se obtiene al descomprimir con el nombre “**IRremoteESP8266**” en la subcarpeta libraries de la carpeta de instalación del IDE de Arduino.

1. Receptor y mando infrarrojo.

CONEXIÓN DEL RECEPTOR IR

* Se debe colocar el receptor en la placa (**mirando hacia delante**) y conectar:

- El pin de la derecha a 3V del nodeMCU.
- El pin central a masa.
- El pin de la izquierda a la patilla del nodeMCU deseada.



1. Receptor y mando infrarrojo.

Enunciado: Programa que conecta un mando infrarrojo a un nodeMCU y muestra en el Monitor Serie la tecla pulsada entre corchetes.

Material:

- * 1 Placa nodeMCU.
- * 2 Placas de pruebas (Breadboard).
- * 1 receptor infrarrojo.
- * 4 cables macho a macho.

```
#include <IRremoteESP8266.h>
#include <IRrecv.h>
```

```
unsigned long claves[21] = {
    0xFFA25D, 0xFF629D, 0xFFE21D, 0xFF22DD, 0xFF02FD, 0xFFC23D, 0xFFE01F,
    0xFFA857, 0xFF906F, 0xFF6897, 0xFF9867, 0xFFB04F, 0xFF30CF, 0xFF18E7,
    0xFF7A85, 0xFF10EF, 0xFF38C7, 0xFF5AA5, 0xFF42BD, 0xFF4AB5, 0xFF52AD
};

String valor[21] = {
    "ON/OFF", "MODE", "VoI OFF", "PLAY/PAUSE", "PREV", "NEXT", "EQ", "-",
    "+", "0", "LOOP", "U/SD", "1", "2", "3", "4", "5", "6", "7", "8", "9"};

int RECV_PIN = 0; // pin izquierda del receptor IR al pin D3 de Arduino

IRrecv irrecv(RECV_PIN); // Crea objeto de 'irrecv'
decode_results results; // Crea objeto de 'decode_results'

void setup() {
    Serial.begin(115200);
    irrecv.enableIRIn(); // Inicializa el receptor
}

void loop() {
    String s;
    if (irrecv.decode(&results)) { // Si se ha recibido alguna señal
        s = getValor(results.value); // Muestra informacion en LCD
        if(s!="") {
            Serial.println( "[" + s + "]" );
        }
        irrecv.resume(); // Recibir el siguiente valor
    }
}
```

```

#include <IRremoteESP8266.h>
#include <IRrecv.h>

unsigned long claves[21] = {
    0xFFA25D, 0xFF629D, 0xFFE21D, 0xFF22DD, 0xFF02FD, 0xFFC23D, 0xFFE01F,
    0xFFA857, 0xFF906F, 0xFF6897, 0xFF9867, 0xFFB04F, 0xFF30CF, 0xFF18E7,
    0xFF7A85, 0xFF10EF, 0xFF38C7, 0xFF5AA5, 0xFF42BD, 0xFF4AB5, 0xFF52AD
};

String valor[21] = {
    "ON/OFF", "MODE", "VoI OFF", "PLAY/PAUSE", "PREV", "NEXT", "EQ", "-",
    "+", "0", "LOOP", "U/SD", "1", "2", "3", "4", "5", "6", "7", "8", "9"};

int RECV_PIN = 0; // pin izquierda del receptor IR al pin D3 de Arduino

IRrecv irrecv(RECV_PIN); // Crea objeto de 'irrecv'
decode_results results; // Crea objeto de 'decode_results'

void setup() {
    Serial.begin(115200);
    irrecv.enableIRIn(); // Inicializa el receptor
}

void loop() {
    String s;
    if (irrecv.decode(&results)) { // Si se ha recibido alguna señal
        s = getValor(results.value); // Muestra informacion en LCD
        if(s!="") {
            Serial.println( "[" + s + "]" );
        }
        irrecv.resume(); // Recibir el siguiente valor
    }
}

```



```

#include <IRremoteESP8266.h>
#include <IRrecv.h>

unsigned long claves[21] = {
    0xFFA25D, 0xFF629D, 0xFFE21D, 0xFF22DD, 0xFF02FD, 0xFFC23D, 0xFFE01F,
    0xFFA857, 0xFF906F, 0xFF6897, 0xFF9867, 0xFFB04F, 0xFF30CF, 0xFF18E7,
    0xFF7A85, 0xFF10EF, 0xFF38C7, 0xFF5AA5, 0xFF42BD, 0xFF4AB5, 0xFF52AD
};

String valor[21] = {
    "ON/OFF", "MODE", "VoI OFF", "PLAY/PAUSE", "PREV", "NEXT", "EQ", "-",
    "+", "0", "LOOP", "U/SD", "1", "2", "3", "4", "5", "6", "7", "8", "9"};

int RECV_PIN = 0; // pin izquierda del receptor IR al pin D3 de Arduino

IRrecv irrecv(RECV_PIN); // Crea objeto de 'irrecv'
decode_results results; // Crea objeto de 'decode_results'

void setup() {
    Serial.begin(115200);
    irrecv.enableIRIn(); // Inicializa el receptor
}

void loop() {
    String s;
    if (irrecv.decode(&results)) { // Si se ha recibido alguna señal
        s = getValor(results.value); // Muestra informacion en LCD
        if(s!="") {
            Serial.println( "[" + s + "]" );
        }
        irrecv.resume(); // Recibir el siguiente valor
    }
}

```

```

#include <IRremoteESP8266.h>
#include <IRrecv.h>

unsigned long claves[21] = {
    0xFFA25D, 0xFF629D, 0xFFE21D, 0xFF22DD, 0xFF02FD, 0xFFC23D, 0xFFE01F,
    0xFFA857, 0xFF906F, 0xFF6897, 0xFF9867, 0xFFB04F, 0xFF30CF, 0xFF18E7,
    0xFF7A85, 0xFF10EF, 0xFF38C7, 0xFF5AA5, 0xFF42BD, 0xFF4AB5, 0xFF52AD
};

String valor[21] = {
    "ON/OFF", "MODE", "VoI OFF", "PLAY/PAUSE", "PREV", "NEXT", "EQ", "-",
    "+", "0", "LOOP", "U/SD", "1", "2", "3", "4", "5", "6", "7", "8", "9"};

int RECV_PIN = 0; // pin izquierda del receptor IR al pin D3 de Arduino

IRrecv irrecv(RECV_PIN); // Crea objeto de 'irrecv'
decode_results results; // Crea objeto de 'decode_results'

void setup() {
    Serial.begin(115200);
    irrecv.enableIRIn(); // Inicializa el receptor
}

void loop() {
    String s;
    if (irrecv.decode(&results)) { // Si se ha recibido alguna señal
        s = getValor(results.value); // Muestra informacion en LCD
        if(s!="") {
            Serial.println( "[" + s + "]" );
        }
        irrecv.resume(); // Recibir el siguiente valor
    }
}

```

```

#include <IRremoteESP8266.h>
#include <IRrecv.h>

unsigned long claves[21] = {
    0xFFA25D, 0xFF629D, 0xFFE21D, 0xFF22DD, 0xFF02FD, 0xFFC23D, 0xFFE01F,
    0xFFA857, 0xFF906F, 0xFF6897, 0xFF9867, 0xFFB04F, 0xFF30CF, 0xFF18E7,
    0xFF7A85, 0xFF10EF, 0xFF38C7, 0xFF5AA5, 0xFF42BD, 0xFF4AB5, 0xFF52AD
};

String valor[21] = {
    "ON/OFF", "MODE", "VoI OFF", "PLAY/PAUSE", "PREV", "NEXT", "EQ", "-",
    "+", "0", "LOOP", "U/SD", "1", "2", "3", "4", "5", "6", "7", "8", "9"};

int RECV_PIN = 0; // pin izquierda del receptor IR al pin D3 de Arduino

IRrecv irrecv(RECV_PIN); // Crea objeto de 'irrecv'
decode_results results; // Crea objeto de 'decode_results'

void setup() {
    Serial.begin(115200);
    irrecv.enableIRIn(); // Inicializa el receptor
}

void loop() {
    String s;
    if (irrecv.decode(&results)) { // Si se ha recibido alguna señal
        s = getValor(results.value); // Muestra informacion en LCD
        if(s!="") {
            Serial.println( "[" + s + "]" );
        }
        irrecv.resume(); // Recibir el siguiente valor
    }
}

```

```

#include <IRremoteESP8266.h>
#include <IRrecv.h>

unsigned long claves[21] = {
    0xFFA25D, 0xFF629D, 0xFFE21D, 0xFF22DD, 0xFF02FD, 0xFFC23D, 0xFFE01F,
    0xFFA857, 0xFF906F, 0xFF6897, 0xFF9867, 0xFFB04F, 0xFF30CF, 0xFF18E7,
    0xFF7A85, 0xFF10EF, 0xFF38C7, 0xFF5AA5, 0xFF42BD, 0xFF4AB5, 0xFF52AD
};

String valor[21] = {
    "ON/OFF", "MODE", "VoI OFF", "PLAY/PAUSE", "PREV", "NEXT", "EQ", "-",
    "+", "0", "LOOP", "U/SD", "1", "2", "3", "4", "5", "6", "7", "8", "9"};

int RECV_PIN = 0; // pin izquierda del receptor IR al pin D3 de Arduino

IRrecv irrecv(RECV_PIN); // Crea objeto de 'irrecv'
decode_results results; // Crea objeto de 'decode_results'

void setup() {
    Serial.begin(115200);
    irrecv.enableIRIn(); // Inicializa el receptor
}

void loop() {
    String s;
    if (irrecv.decode(&results)) { // Si se ha recibido alguna señal
        s = getValor(results.value); // Muestra informacion en LCD
        if(s!="") {
            Serial.println( "[" + s + "]" );
        }
        irrecv.resume(); // Recibir el siguiente valor
    }
}

```

```

#include <IRremoteESP8266.h>
#include <IRrecv.h>

unsigned long claves[21] = {
    0xFFA25D, 0xFF629D, 0xFFE21D,
    0xFFA857, 0xFF906F, 0xFF6897,
    0xFF7A85, 0xFF10EF, 0xFF38C7,
};

String valor[21] = {
    "ON/OFF", "MODE", "VoLOFF", "PLA",
    "+", "0", "LOOP", "U/SD", "1",
};

int RECV_PIN = 0; // pin izquierda del receptor IR al pin D3 de Arduino

IRrecv irrecv(RECV_PIN); // Crea objeto de 'irrecv'
decode_results results; // Crea objeto de 'decode_results'

void setup() {
    Serial.begin(115200);
    irrecv.enableIRIn(); // Inicializa el receptor
}

void loop() {
    String s;
    if (irrecv.decode(&results)) { // Si se ha recibido alguna señal
        s = getValor(results.value); // Muestra informacion en LCD
        if(s!="") {
            Serial.println( "[" + s + "]" );
        }
        irrecv.resume(); // Recibir el siguiente valor
    }
}

```

```

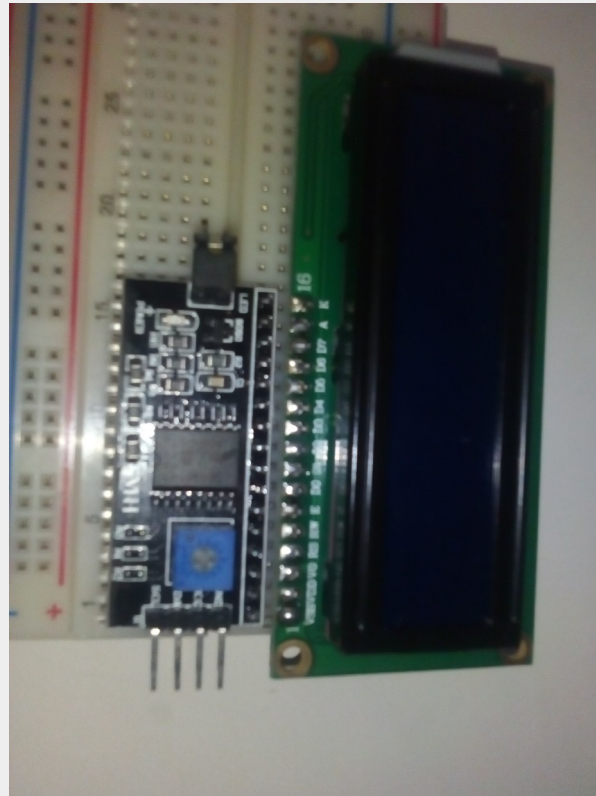
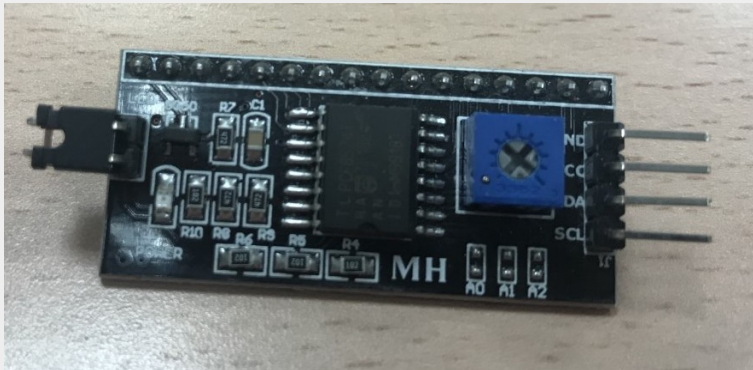
String getValor(unsigned long c) {
    byte i;
    i=0;
    while( (i<21) && (claves[i]!=c) ) {
        i++;
    }
    if (i==21) return String("");
    else return String(valor[i]);
}

```


2. Conexión LCD.

CONEXIÓN DEL LCD

- * GND a cualquier G del nodeMCU
- * VCC a VU del nodeMCU. Esta salida del nodeMCU da 5 voltios de salida.
- * SDA se conectará al pin seleccionado del lcd, en nuestro caso el pin 5 (D1).
- * SCL se conectará al pin seleccionado del lcd, en nuestro caso el pin 4 (D2).



2. Conexión LCD.

INSTALACIÓN DE LIBRERÍAS

- * Es necesario añadir la librería LiquidCrystal_I2C.h al IDE de Arduino.
- * Los pasos necesarios para añadir la librería son los siguientes: Bajar de la dirección **https://github.com/marcoschwartz/LiquidCrystal_I2C** el archivo zip que contiene la librería y copiar la carpeta que se obtiene al descomprimir con el nombre “**LiquidCrystal_I2C.h**” en la subcarpeta libraries de la carpeta de instalación del IDE de Arduino.

```
#include <LiquidCrystal_I2C.h> // Necesaria para el lcd
#include <Wire.h> // Permite conectar los pines del interfaz I2C
```

```
LiquidCrystal_I2C lcd(0x27, 16, 2); // Crea objeto para lcd
//LiquidCrystal_I2C lcd(0x3F, 16, 2); // Crea objeto para lcd
```

```
byte turno; // Variable que selecciona fila a escribir
```

```
void setup() {
    turno = 1;
    Wire.begin(5, 4); // Use predefined PINS consts // conecta el I2C a pines 5 y 4
    lcd.setBacklight(HIGH); // Activa luz del fondo del lcd
    lcd.begin(16,2); // Inicia lcd indicando n° de columnas y filas
    lcd.home(); // Coloca el cursor en la esquina superior izquierda del lcd
    lcd.setCursor(0,1); // Posiciona el cursor en lcd (col 1 y fila 0)
    lcd.print("Mundo hola (1)"); // Muestra texto en posicion actual del cursor
}
```

```
void loop() {
    lcd.clear(); // limpia el lcd
    if (turno==1) {
        lcd.setCursor(0,1); // Posiciona el cursor en fila 1
        lcd.print("Mundo hola (1)"); // Muestra texto en posicion cursor
        turno = 0; // Actualizar variable de cambio de fila
    } else {
        lcd.setCursor(0,0); // Posiciona el cursor en fila 0
        lcd.print("Hola mundo (0)"); // Muestra texto en posicion cursor
        turno = 1; // Actualizar variable de cambio de fila
    }
    delay(2000);
}
```



```
#include <LiquidCrystal_I2C.h> // Necesaria para el lcd
#include <Wire.h> // Permite conectar los pines del interfaz I2C
```

```
LiquidCrystal_I2C lcd(0x27, 16, 2); // Crea objeto para lcd
//LiquidCrystal_I2C lcd(0x3F, 16, 2); // Crea objeto para lcd
```

```
byte turno; // Variable que selecciona fila a escribir
```

```
void setup() {
    turno = 1;
    Wire.begin(5, 4); // Use predefined PINS consts // conecta el I2C a pines 5 y 4
    lcd.setBacklight(HIGH); // Activa luz del fondo del lcd
    lcd.begin(16,2); // Inicia lcd indicando n° de columnas y filas
    lcd.home(); // Coloca el cursor en la esquina superior izquierda del lcd
    lcd.setCursor(0,1); // Posiciona el cursor en lcd (col 1 y fila 0)
    lcd.print("Mundo hola (1)"); // Muestra texto en posicion actual del cursor
}
```

```
void loop() {
    lcd.clear(); // limpia el lcd
    if (turno==1) {
        lcd.setCursor(0,1); // Posiciona el cursor en fila 1
        lcd.print("Mundo hola (1)"); // Muestra texto en posicion cursor
        turno = 0; // Actualizar variable de cambio de fila
    } else {
        lcd.setCursor(0,0); // Posiciona el cursor en fila 0
        lcd.print("Hola mundo (0)"); // Muestra texto en posicion cursor
        turno = 1; // Actualizar variable de cambio de fila
    }
    delay(2000);
}
```

```

#include <LiquidCrystal_I2C.h> // Necesaria para el lcd
#include <Wire.h> // Permite conectar los pines del interfaz I2C

LiquidCrystal_I2C lcd(0x27, 16, 2); // Crea objeto para lcd
//LiquidCrystal_I2C lcd(0x3F, 16, 2); // Crea objeto para lcd

byte turno; // Variable que selecciona fila a escribir

void setup() {
    turno = 1;
    Wire.begin(5, 4); // Use predefined PINS consts // conecta el I2C a pines 5 y 4
    lcd.setBacklight(HIGH); // Activa luz del fondo del lcd
    lcd.begin(16,2); // Inicia lcd indicando nº de columnas y filas
    lcd.home(); // Coloca el cursor en la esquina superior izquierda del lcd
    lcd.setCursor(0,1); // Posiciona el cursor en lcd (col 1 y fila 0)
    lcd.print("Mundo hola (1)"); // Muestra texto en posicion actual del cursor
}

void loop() {
    lcd.clear(); // limpia el lcd
    if (turno==1) {
        lcd.setCursor(0,1); // Posiciona el cursor en fila 1
        lcd.print("Mundo hola (1)"); // Muestra texto en posicion cursor
        turno = 0; // Actualizar variable de cambio de fila
    } else {
        lcd.setCursor(0,0); // Posiciona el cursor en fila 0
        lcd.print("Hola mundo (0)"); // Muestra texto en posicion cursor
        turno = 1; // Actualizar variable de cambio de fila
    }
    delay(2000);
}

```

```
#include <LiquidCrystal_I2C.h> // Necesaria para el lcd
#include <Wire.h> // Permite conectar los pines del interfaz I2C

LiquidCrystal_I2C lcd(0x27, 16, 2); // Crea objeto para lcd
//LiquidCrystal_I2C lcd(0x3F, 16, 2); // Crea objeto para lcd

byte turno; // Variable que selecciona fila a escribir
```

```
void setup() {
    turno = 1;
    Wire.begin(5, 4); // Use predefined PINS consts // conecta el I2C a pines 5 y 4
    lcd.setBacklight(HIGH); // Activa luz del fondo del lcd
    lcd.begin(16,2); // Inicia lcd indicando n° de columnas y filas
    lcd.home(); // Coloca el cursor en la esquina superior izquierda del lcd
    lcd.setCursor(0,1); // Posiciona el cursor en lcd (col 1 y fila 0)
    lcd.print("Mundo hola (1)"); // Muestra texto en posicion actual del cursor
}
```

```
void loop() {
    lcd.clear(); // limpia el lcd
    if (turno==1) {
        lcd.setCursor(0,1); // Posiciona el cursor en fila 1
        lcd.print("Mundo hola (1)"); // Muestra texto en posicion cursor
        turno = 0; // Actualizar variable de cambio de fila
    } else {
        lcd.setCursor(0,0); // Posiciona el cursor en fila 0
        lcd.print("Hola mundo (0)"); // Muestra texto en posicion cursor
        turno = 1; // Actualizar variable de cambio de fila
    }
    delay(2000);
}
```

```

#include <LiquidCrystal_I2C.h> // Necesaria para el lcd
#include <Wire.h> // Permite conectar los pines del interfaz I2C

LiquidCrystal_I2C lcd(0x27, 16, 2); // Crea objeto para lcd
//LiquidCrystal_I2C lcd(0x3F, 16, 2); // Crea objeto para lcd

byte turno; // Variable que selecciona fila a escribir

void setup() {
    turno = 1;
    Wire.begin(5, 4); // Use predefined PINS consts // conecta el I2C a pines 5 y 4
    lcd.setBacklight(HIGH); // Activa luz del fondo del lcd
    lcd.begin(16,2); // Inicia lcd indicando nº de columnas y filas
    lcd.home(); // Coloca el cursor en la esquina superior izquierda del lcd
    lcd.setCursor(0,1); // Posiciona el cursor en lcd (col 1 y fila 0)
    lcd.print("Mundo hola (1)"); // Muestra texto en posicion actual del cursor
}

void loop() {
    lcd.clear(); // limpia el lcd
    if (turno==1) {
        lcd.setCursor(0,1); // Posiciona el cursor en fila 1
        lcd.print("Mundo hola (1)"); // Muestra texto en posicion cursor
        turno = 0; // Actualizar variable de cambio de fila
    } else {
        lcd.setCursor(0,0); // Posiciona el cursor en fila 0
        lcd.print("Hola mundo (0)"); // Muestra texto en posicion cursor
        turno = 1; // Actualizar variable de cambio de fila
    }
    delay(2000);
}

```

```

#include <LiquidCrystal_I2C.h> // Necesaria para el lcd
#include <Wire.h> // Permite conectar los pines del interfaz I2C

LiquidCrystal_I2C lcd(0x27, 16, 2); // Crea objeto para lcd
//LiquidCrystal_I2C lcd(0x3F, 16, 2); // Crea objeto para lcd

byte turno; // Variable que selecciona fila a escribir

void setup() {
    turno = 1;
    Wire.begin(5, 4); // Use predefined PINS consts // conecta el I2C a pines 5 y 4
    lcd.setBacklight(HIGH); // Activa luz del fondo del lcd
    lcd.begin(16,2); // Inicia lcd indicando n° de columnas y filas
    lcd.home(); // Coloca el cursor en la esquina superior izquierda del lcd
    lcd.setCursor(0,1); // Posiciona el cursor en lcd (col 1 y fila 0)
    lcd.print("Mundo hola (1)"); // Muestra texto en posicion actual del cursor
}

void loop() {
    lcd.clear(); // limpia el lcd
    if (turno==1) {
        lcd.setCursor(0,1); // Posiciona el cursor en fila 1
        lcd.print("Mundo hola (1)"); // Muestra texto en posicion cursor
        turno = 0; // Actualizar variable de cambio de fila
    } else {
        lcd.setCursor(0,0); // Posiciona el cursor en fila 0
        lcd.print("Hola mundo (0)"); // Muestra texto en posicion cursor
        turno = 1; // Actualizar variable de cambio de fila
    }
    delay(2000);
}

```

```

#include <LiquidCrystal_I2C.h> // Necesaria para el lcd
#include <Wire.h> // Permite conectar los pines del interfaz I2C

LiquidCrystal_I2C lcd(0x27, 16, 2); // Crea objeto para lcd
//LiquidCrystal_I2C lcd(0x3F, 16, 2); // Crea objeto para lcd

byte turno; // Variable que selecciona fila a escribir

void setup() {
    turno = 1;
    Wire.begin(5, 4); // Use predefined PINS consts // conecta el I2C a pines 5 y 4
    lcd.setBacklight(HIGH); // Activa luz del fondo del lcd
    lcd.begin(16,2); // Inicia lcd indicando nº de columnas y filas
    lcd.home(); // Coloca el cursor en la esquina superior izquierda del lcd
    lcd.setCursor(0,1); // Posiciona el cursor en lcd (col 1 y fila 0)
    lcd.print("Mundo hola (1)"); // Muestra texto en posicion actual del cursor
}

void loop() {
    lcd.clear(); // limpia el lcd
    if (turno==1) {
        lcd.setCursor(0,1); // Posiciona el cursor en fila 1
        lcd.print("Mundo hola (1)"); // Muestra texto en posicion cursor
        turno = 0; // Actualizar variable de cambio de fila
    } else {
        lcd.setCursor(0,0); // Posiciona el cursor en fila 0
        lcd.print("Hola mundo (0)"); // Muestra texto en posicion cursor
        turno = 1; // Actualizar variable de cambio de fila
    }
    delay(2000);
}

```



```

#include <LiquidCrystal_I2C.h> // Necesaria para el lcd
#include <Wire.h> // Permite conectar los pines del interfaz I2C

LiquidCrystal_I2C lcd(0x27, 16, 2); // Crea objeto para lcd
//LiquidCrystal_I2C lcd(0x3F, 16, 2); // Crea objeto para lcd

byte turno; // Variable que selecciona fila a escribir

void setup() {
    turno = 1;
    Wire.begin(5, 4); // Use predefined PINS consts // conecta el I2C a pines 5 y 4
    lcd.setBacklight(HIGH); // Activa luz del fondo del lcd
    lcd.begin(16,2); // Inicia lcd indicando nº de columnas y filas
    lcd.home(); // Coloca el cursor en la esquina superior izquierda del lcd
    lcd.setCursor(0,1); // Posiciona el cursor en lcd (col 1 y fila 0)
    lcd.print("Mundo hola (1)"); // Muestra texto en posicion actual del cursor
}

void loop() {
    lcd.clear(); // limpia el lcd
    if (turno==1) {
        lcd.setCursor(0,1); // Posiciona el cursor en fila 1
        lcd.print("Mundo hola (1)"); // Muestra texto en posicion cursor
        turno = 0; // Actualizar variable de cambio de fila
    } else {
        lcd.setCursor(0,0); // Posiciona el cursor en fila 0
        lcd.print("Hola mundo (0)"); // Muestra texto en posicion cursor
        turno = 1; // Actualizar variable de cambio de fila
    }
    delay(2000);
}

```

3. Programas ejemplo.

Enunciado: programa que desplaza un 0 en la fila 0 y un 1 en la fila 1 a lo largo de la misma alternativamente.

Material: El hardware necesario es el siguiente:

- * 1 Placa nodeMCU.
- * 2 Placas de pruebas (Breadboard).
- * 1 LCD I2C (si no es I2C se necesita un adaptador).
- * Cables macho a macho y hembra hembra.


```

#include <LiquidCrystal_I2C.h>
#include <Wire.h>

LiquidCrystal_I2C lcd(0x27, 16, 2);

int fila = 0; // Indica fila a usar
int pos = 0; // Posicion dentro de la fila

void setup() {
    Wire.begin(5, 4); // Pines del nodeMCU a usar
    lcd.setBacklight(HIGH);
    lcd.begin(16,2);
    lcd.home();
    lcd.setCursor(pos, fila); // Posicion inicial del cursor
    lcd.print(""); // Mensaje inicial
}

void loop() {
    lcd.clear(); // Limpia LCD
    if (fila==0) { // Si fila 0
        lcd.setCursor(pos,0); // posiciona cursor
        lcd.print("0"); // muestra un 0
        fila = 1; // Cambio a fila 1
    } else {
        lcd.setCursor(pos,1); // posiciona cursor
        lcd.print("1"); // muestra un 1
        fila = 0; // Cambio a fila 1
        pos = (pos+1) % 16; // Incrementa y si es 16 pasa a 0
    }
    delay(1000); // Retardo para visualizar
}

```

3. Programas ejemplo.

Enunciado: programa muestra el uso conjunto del mando y receptor IR junto con el lcd en el nodeMCU. El montaje del lcd es el mismo que en el caso anterior. Además, el montaje del receptor y mando IR es igual que el presentado anteriormente, utilizando el pin 0 del nodeMCU en este caso.

Material: El hardware necesario es el siguiente:

- * 1 Placa nodeMCU.
- * 2 Placas de pruebas (Breadboard).
- * 1 receptor infrarrojo.
- * 1 LCD I2C (si no es I2C se necesita un adaptador).
- * Cables macho a macho y hembra hembra.

```

#include <IRremoteESP8266.h>
#include <IRrecv.h>

#include <LiquidCrystal_I2C.h>
#include <Wire.h>

LiquidCrystal_I2C lcd(0x27, 16, 2);

unsigned long claves[21] = {
    0xFFA25D, 0xFF629D, 0xFFE21D, 0xFF22DD, 0xFF02FD, 0xFFC23D, 0xFFE01F,
    0xFFA857, 0xFF906F, 0xFF6897, 0xFF9867, 0xFFB04F, 0xFF30CF, 0xFF18E7,
    0xFF7A85, 0xFF10EF, 0xFF38C7, 0xFF5AA5, 0xFF42BD, 0xFF4AB5, 0xFF52AD};

String valor[21] = {
    "ON/OFF", "MODE", "Vo1OFF", "PLAY/PAUSE", "PREV", "NEXT", "EQ", "-",
    "+", "0", "LOOP", "U/SD", "1", "2", "3", "4", "5", "6", "7", "8", "9"};

int RECV_PIN = 0; // pin del receptor IR al pin 0 del nodeMCU

IRrecv irrecv(RECV_PIN); // Crea objeto de 'irrecv'
decode_results results; // Crea objeto de 'decode_results'

void setup() {
    Serial.begin(115200);
    irrecv.enableIRIn(); // Inicializa el receptor
    Wire.begin(5, 4);
    lcd.setBacklight(HIGH); //Use predefined PINS consts
    lcd.begin(16,2);
    lcd.home();
    lcd.setCursor(0,1); // Situa el cursor en 2ª Fila
    lcd.print("Practicas II"); // Muestra mensaje inicial
}

```

```

void loop() {
    String s;
    if (irrecv.decode(&results)) { // Si se ha recibido alguna señal
        s = getValor(results.value); // Recoge cadena desde código
        if(s!="") {
            Serial.println( "[" + s + "]" );
            escribeLCD(s); // Escribo en LCD
        }
        irrecv.resume(); // Recibir el siguiente valor
    }
}

```

```

void escribeLCD(String s) {
    lcd.clear(); // Limpia el LCD
    if(s!="") {
        lcd.setCursor(0,0); // Coloca el cursor en la fila 0
        lcd.print(s);
    } else {
        lcd.setCursor(0,1); // Coloca cursor en fila 1
        lcd.print( "--- Rebote ---"); // Muestra mensaje rebote
    }
}

```

```

String getValor(unsigned long c) {
    byte i;
    i=0;
    while( (i<21) && (claves[i]!=c) ) {
        i++;
    }
    if (i==21) return String("");
    else return String(valor[i]);
}

```

3. Programas ejemplo.

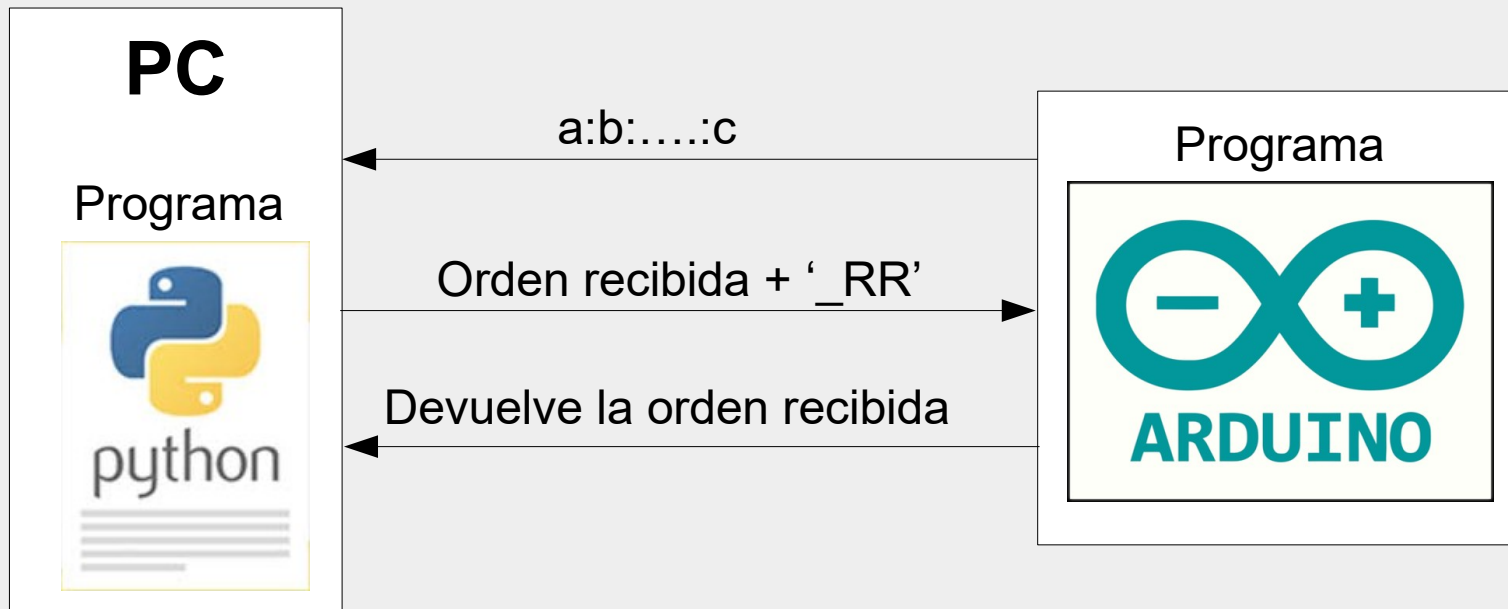
Enunciado: programa en el que el nodeMCU hace de cliente recibiendo las órdenes mediante un mando infrarrojo. Recibe un comando y lo envía al programa *Python* que lo recibe y le añade al final '_RR' (ése será su “servicio”). Finalmente se lo devuelve al *nodeMCU* que le devuelve el mensaje recibido a modo de reconocimiento.

Material: El hardware necesario es el siguiente:

- * 1 Placa nodeMCU.
- * 2 Placas de pruebas (Breadboard).
- * 1 receptor infrarrojo.
- * 1 LCD I2C (si no es I2C se necesita un adaptador).
- * Cables macho a macho y hembra hembra.

5. Ejemplos.

EJEMPLO: programa en el que el nodeMCU hace de cliente recibiendo las órdenes mediante un mando infrarrojo con el formato “a:b:....:c”. Recibe la orden del infrarrojo y la envía al programa Python que la recibe y le añade al final ‘_RR’ (ése será su “servicio”). Finalmente se lo devuelve al nodeMCU que le devuelve el mensaje recibido a modo de reconocimiento..



```
#!/usr/bin/python
```

```
# Importamos la libreria de PySerial
```

```
import serial
```

```
from mensaje import *
```

```
# Abrimos el puerto del arduino a 115200
```

```
#PuertoSerie = abrirPort( '/dev/ttyUSB0' )
```

```
PuertoSerie = abrirPort( 'COM5', 115200 )
```

```
# Creamos un bucle sin fin
```

```
while True:
```

```
    print("Esperando....")
```

```
    r = receiveMensaje(PuertoSerie)
```

```
    print( 'Respuesta recibida: ', r )
```

```
    # añade "_RR" y envía
```

```
    sendMensaje(PuertoSerie,r+'_RR')
```

```
    # Espera mensaje de confirmación
```

```
    r1 = receiveMensaje(PuertoSerie)
```

```
    print( 'Respuesta recibida segunda: ', r1 )
```



```
#include <IRremoteESP8266.h>
#include <IRrecv.h>

unsigned long claves[21] = {
    0xFFA25D, 0xFF629D, 0xFFE21D, 0xFF22DD, 0xFF02FD, 0xFFC23D, 0xFFE01F,
    0xFFA857, 0xFF906F, 0xFF6897, 0xFF9867, 0xFFB04F, 0xFF30CF, 0xFF18E7,
    0xFF7A85, 0xFF10EF, 0xFF38C7, 0xFF5AA5, 0xFF42BD, 0xFF4AB5, 0xFF52AD};

String valor[21] = {
    "ON/OFF", "MODE", "VolOFF", "PLAY/PAUSE", "PREV", "NEXT", "EQ", "-",
    "+", "0", "LOOP", "U/SD", "1", "2", "3", "4", "5", "6", "7", "8", "9"};

int RECV_PIN = 0; // pin OUT del receptor IR a D3 de Arduino

IRrecv irrecv(RECV_PIN); // Crea objeto de 'irrecv'
decode_results results; // Crea objeto de 'decode_results'

void setup() {
    Serial.begin(115200);
    irrecv.enableIRIn(); // Inicializa el receptor
}
```

3. Programas ejemplo.

```
void loop() {  
    String s, r = "";  
    if (irrecv.decode(&results)) { // Si se ha recibido alguna señal  
  
        // Lee el código pulsado y envía a programa python  
        s = leerHastaRetorno();  
        Serial.println( s );  
  
        // Recibir respuesta  
        r = leerLinea(); // recibe info hasta '\n'  
  
        // Envía lo recibido, terminando en "\r\n", añadido por el println  
        Serial.println( r ); // devolver respuesta  
    }  
}
```

3. Programas ejemplo.

```
void loop()  
  String s,  
  if (irrecv  
  
  // Lee  
  s = lee  
  Serial.  
  
  // Recibi  
  r = lee  
  
  // Enví  
  Serial.  
}  
}
```

```
clienteIR $ infrarrojo lectura  
// Lee pulsaciones del mando hasta que se pulsa "U/SD"  
String leerHastaRetorno() {  
  bool b = false;  
  String r="", s;  
  while(!b) {  
    if (irrecv.decode(&results)) { // Si se ha recibido alguna señal  
      // Lee el código pulsado y envía a programa python  
      s = getValor(results.value);  
      if(s!="U/SD") {  
        // Enviar identificador pulsado a python  
        if(s=="LOOP") r = r + String(":");  
        else r = r + s;  
      } else {  
        b = true; // completada cadena  
      }  
      irrecv.resume(); // Recibir el siguiente valor  
    }  
    // Necesario para evitar el problema del "Soft WDT reset"  
    // caused by watchdog timer.  
    delay(1); //  
  }  
  return String(r);  
}
```

3. Programas ejemplo.

```
void loop() {  
    String s, r = "";  
    if (irrecv.decode(&results)) { // Si se ha recibido alguna señal  
  
        // Lee el código pulsado y envía a programa python  
        s = leerHastaRetorno();  
        Serial.println( s );  
  
        // Recibir respuesta  
        r = leerLinea(); // recibe info hasta '\n'  
  
        // Envía lo recibido, terminando en "\r\n", añadido por el println  
        Serial.println( r ); // devolver respuesta  
    }  
}
```

3. Programas ejemplo.

```
void loop() {  
    String s, r = "";  
    if (irrecv.decode(&results)) { // Si se ha recibido alguna señal  
  
        // Lee el código pulsado y envía a programa python  
        s = leerHastaRetorno();  
        Serial.println( s );  
  
        // Recibir respuesta  
        r = leerLinea(); // recibe info hasta '\n'  
  
        // Envía lo recibido, terminando en "\r\n", añadido por el println  
        Serial.println( r ); // devolver respuesta  
    }  
}
```

2. Programa a realizar.

Enunciado: .

Material:

- * 1 Placa nodeMCU.
- * 2 Placas de pruebas (Breadboard).
- * 1 PCF8574 de 8 puertos.
- * 4 cables macho a macho y 4 cables hembra.

3. Mejoras.

* Cambiar el programa para que los modos activen con los Strings '**' y '##'. En este caso, los caracteres válidos serán del 0 al 9 y '*' para el modo numérico, y A, B, C, D y '#' para el modo carácter.